Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Struktur Kontrol dan Percabangan

Pertemuan 11

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 3.3

Mengimplementasikan struktur kontrol (`CMP`, `TEST`, lompatan bersyarat `Jcc`, `LOOP`, dan prosedur dengan `CALL`/`RET`) dalam program assembly.

CPMK yang didukung: CPMK03 ▪ **CPL yang didukung: CPL03**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan peran *flag* sebagai dasar keputusan;
2. membedakan cara kerja `CMP` dan `TEST`;
3. memilih `Jcc` untuk bilangan bertanda dan tak bertanda;
4. menjelaskan pengaruh jarak lompatan pada ukuran kode;
5. menyusun perulangan `LOOP` dan membandingkannya dengan `DEC` serta `JNZ`;
6. menerjemahkan pola `if`, `while`, `for`, dan `switch` ke assembly.

Peta Materi

1. **Flag sebagai Dasar Pengambilan Keputusan**
   lima *flag* utama; `CMP` dan `TEST` sebagai penyiap *flag*; lompatan tak bersyarat `JMP` dan jarak lompatan

2. **Lompatan Bersyarat dan Perulangan**
   keluarga `Jcc`; pemilihan lompatan untuk bilangan bertanda dan tak bertanda; label dan kesalahan umum; perulangan `LOOP` versus `DEC` dan `JNZ`

3. **Penerjemahan Pola Bahasa Tingkat Tinggi**
   pola `if`, `if-else`, `while`, `do-while`, `for`, dan `switch`; studi kasus maksimum, pengurutan gelembung, pencarian biner

Praktik pada pertemuan ini

Menelusuri `loop-maksimum.asm` dan `bubble-sort.asm` dengan *Single Step*, lalu mencatat keadaan *flag* tepat sebelum setiap lompatan bersyarat.

BAGIAN

Flag sebagai Dasar Pengambilan Keputusan

Mengapa Prosesor Membutuhkan Flag

8086 tidak memiliki instruksi "jika"; yang tersedia adalah **flag** — penanda keadaan pada `FLAGS` — dan instruksi lompatan yang memeriksanya. Pengambilan keputusan berlangsung dalam **dua langkah**: satu instruksi mengubah *flag*, lalu satu lompatan bersyarat memutuskan arah eksekusi.
Penting: Langkah kedua tidak menghitung apa pun; lompatan membaca *flag* sisa operasi sebelumnya.

Flag	Makna	Dipakai oleh
`ZF`	Bernilai 1 bila hasil operasi sama dengan nol	`JE`, `JNE`, `JG`, `JL`, `LOOPE`
`CF`	Bernilai 1 bila terjadi pinjaman atau limpahan pada operasi tak bertanda	`JB`, `JAE`, `JC`, `JNC`
`SF`	Menyalin bit paling kiri (bit tanda) hasil operasi	`JS`, `JNS`, serta lompatan bertanda
`OF`	Bernilai 1 bila hasil melampaui jangkauan bilangan bertanda	`JO`, `JNO`, `JG`, `JL`
`PF`	Menyatakan kegenapan banyak bit 1 pada byte hasil	`JP`, `JNP`

Sumber: Bab XII, Mengapa Prosesor Membutuhkan Flag

Instruksi CMP: Menyiapkan Flag melalui Pengurangan

CMP (*compare*) melakukan pengurangan antara operand pertama dan operand kedua, tetapi *tidak menyimpan* hasilnya; yang tersimpan hanyalah perubahan pada *flag*. Bentuk dan aturan operandnya sama persis dengan SUB.

```
MOV AX, 0005h        ; AX = 5
MOV BX, 0003h        ; BX = 3
CMP AX, BX           ; hitung 5 - 3 = 2, hasil dibuang
; keadaan flag: ZF = 0, SF = 0, CF = 0, OF = 0
JG  LEBIH_BESAR      ; lompat karena 5 > 3 (bertanda)
```

Bila kedua operand sama, ZF bernilai 1. Bila operand pertama lebih kecil secara tak bertanda, terjadi pinjaman sehingga CF bernilai 1. Perilaku inilah yang menjadi dasar seluruh tabel lompatan bersyarat.

Mengapa disebut penyiap flag

Isi AX dan BX tidak berubah setelah CMP; yang berubah hanya FLAGS.

Instruksi TEST: Menyiapkan Flag melalui AND

TEST melakukan operasi AND bit demi bit, lalu membuang hasilnya dan hanya menyimpan *flag*. Berbeda dari CMP, TEST tidak melakukan pengurangan; ia *menyaring* bit.

```
TEST AL, 01h        ; uji bit ke-0 dari AL
JZ   GENAP          ; ZF = 1 berarti bit ke-0 nol, bilangan genap

TEST AL, AL         ; uji apakah AL sama dengan nol
JZ   NOL
```

Pemakaian TEST AL, AL adalah idiom yang sering muncul: operand dibandingkan dengan dirinya sendiri sehingga ZF dan SF mengikuti nilai operand, tanpa mengubah isi AL.

Sumber: Bab XII, Instruksi TEST

Perbedaan CMP dan TEST

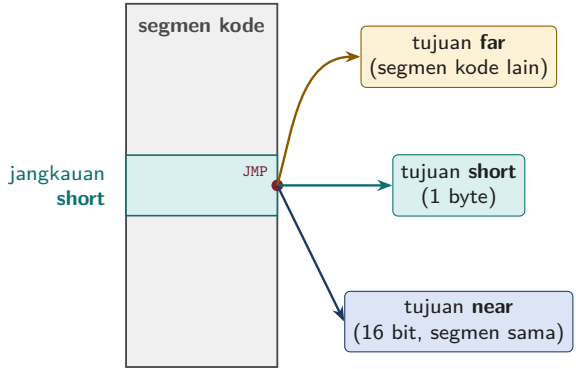
Aspek	`CMP`	`TEST`
Operasi dasar	Pengurangan (*dst* − *src*)	`AND` bit demi bit (*dst* & *src*)
Hasil disimpan	Tidak; hanya *flag*	Tidak; hanya *flag*
Flag utama	`ZF`, `CF`, `SF`, `OF`, `AF`, `PF`	`ZF`, `SF`, `PF`; `CF` dan `OF` selalu 0
Kegunaan khas	Membandingkan dua nilai, menentukan urutan	Menguji bit, memeriksa nol atau tanda
Contoh	`CMP AX, BX`	`TEST AL, 80h`

Kekeliruan yang sering terjadi

`TEST AX, BX` tidak memberi tahu apakah `AX` lebih besar dari `BX`; ia hanya memberi tahu apakah *ada* bit yang bernilai 1 pada kedua operand secara bersamaan. Untuk membandingkan nilai, gunakan `CMP`.

Lompatan Tak Bersyarat JMP dan Jarak Lompatan

`JMP` memindahkan kendali tanpa syarat, misalnya `JMP HITUNG`. Prosesor 8086 tidak menyimpan alamat mutlak tujuan, melainkan **perpindahan** (*displacement*) relatif terhadap instruksi berikutnya; besar perpindahan inilah yang menentukan **jarak lompatan**.



Pengaruh Jarak Lompatan pada Ukuran Kode

Jenis	Jangkauan	Perpindahan	Ukuran	Penulisan
short	$\pm$ 127 byte dari instruksi berikutnya	1 byte bertanda	2 byte	`JMP SHORT L1`
near	Seluruh segmen kode (64 KB)	16 bit	3 byte	`JMP L1`
far	Segmen kode mana pun	`CS:IP` lengkap	5 byte	`JMP FAR PTR L1`

Assembler memilih jarak lompatan secara otomatis; operator `SHORT` atau `FAR PTR` memaksa bentuk tertentu. Bila programmer memaksa `SHORT` padahal jaraknya terlalu jauh, assembler melaporkan kesalahan `Relative jump out of range`.

Mengapa ini penting

Pada program besar — misalnya rutin BIOS yang harus muat dalam 64 KB — pemilihan jarak lompatan dapat menghemat ribuan byte.

Sumber: Bab XII, Lompatan Tak Bersyarat JMP dan Jarak Lompatan

Rangkuman Bagian 1

- Pengambilan keputusan berjalan dalam dua langkah: satu instruksi menyiapkan *flag*, lalu satu lompatan bersyarat membacanya. Lompatan tidak menghitung apa pun.
- Lima *flag* terpenting adalah `ZF`, `CF`, `SF`, `OF`, dan `PF` pada register `FLAGS`.
- `CMP` menyiapkan *flag* melalui pengurangan yang hasilnya dibuang, sedangkan `TEST` melalui `AND` yang hasilnya dibuang. `CMP` membandingkan nilai; `TEST` memeriksa bit.
- `JMP` memindahkan kendali tanpa syarat dan menyimpan perpindahan relatif, bukan alamat mutlak.
- Jarak lompatan menentukan ukuran instruksi: *short* 2 byte ($\pm$ 127 byte), *near* 3 byte (satu segmen), dan *far* 5 byte (antarsegmen).

BAGIAN

Lompatan Bersyarat dan Perulangan

Anatomi Instruksi Jcc

Nama Jcc adalah singkatan dari *jump if condition code*, yaitu seluruh anggota keluarga lompatan bersyarat. Pola penulisannya seragam: mnemonic diikuti satu operand berupa label tujuan.

```
CMP AX, BX          ; siapkan flag terlebih dahulu
JE  SAMA            ; bila ZF = 1, lompat ke label SAMA
JNE TIDAK_SAMA      ; bila ZF = 0, lompat ke label TIDAK_SAMA
```

Pola ini menjelaskan mengapa CMP hampir selalu muncul berpasangan dengan Jcc: CMP menyiapkan *flag*, Jcc membacanya.

Selalu berbentuk short

Berbeda dengan JMP, lompatan bersyarat pada 8086 selalu berbentuk *short*, sehingga jangkauannya terbatas $\pm$ 127 byte.

Tabel Lengkap Jcc: Kesamaan dan Perbandingan

Instruksi	Arti	Syarat flag	Jenis bilangan
JE / JZ	Sama dengan	ZF = 1	Bertanda maupun tak bertanda
JNE / JNZ	Tidak sama dengan	ZF = 0	Bertanda maupun tak bertanda
JG / JNLE	Lebih besar dari	ZF = 0 dan SF = OF	Bertanda
JGE / JNL	Lebih besar atau sama	SF = OF	Bertanda
JL / JNGE	Lebih kecil dari	SF ≠ OF	Bertanda
JLE / JNG	Lebih kecil atau sama	ZF = 1 atau SF ≠ OF	Bertanda
JA / JNBE	Di atas (*above*)	CF = 0 dan ZF = 0	Tak bertanda
JAE / JNB	Di atas atau sama	CF = 0	Tak bertanda
JB / JNAE	Di bawah (*below*)	CF = 1	Tak bertanda
JBE / JNA	Di bawah atau sama	CF = 1 atau ZF = 1	Tak bertanda

Sumber: Bab XII, Tabel Lengkap Lompatan Bersyarat

Tabel Lengkap Jcc: Pemeriksa Flag Tunggal

Instruksi	Arti	Syarat flag	Jenis bilangan
`JC`	Ada *carry*	`CF` $= 1$	Umum
`JNC`	Tidak ada *carry*	`CF` $= 0$	Umum
`JS`	Bertanda negatif	`SF` $= 1$	Bertanda
`JNS`	Bertanda positif	`SF` $= 0$	Bertanda
`JO`	Ada limpahan	`OF` $= 1$	Bertanda
`JNO`	Tidak ada limpahan	`OF` $= 0$	Bertanda
`JP` / `JPE`	Kegenapan genap	`PF` $= 1$	Umum
`JNP` / `JPO`	Kegenapan ganjil	`PF` $= 0$	Umum
`JCXZ`	`CX` bernilai nol	`CX` $= 0$ (bukan flag)	Umum

Beberapa nama, satu opcode

`JAE`, `JNB`, dan `JNC` dirakit menjadi opcode identik, demikian pula `JB`, `JNAE`, dan `JC`. Perbedaan nama semata-mata untuk memperjelas maksud: pilih `JB` ketika membandingkan nilai dan `JC` ketika memeriksa hasil aritmatika.

Memilih Jcc: Bertanda versus Tak Bertanda

Kesalahan paling sering berasal dari sifat *two's complement*: nilai bit `FFh` dapat dibaca sebagai 255 (tak bertanda) atau -1 (bertanda).

```
MOV AL, 0FFh          ; AL = 255 (tak bertanda) atau -1 (bertanda)
MOV BL, 01h           ; BL = 1
CMP AL, BL            ; AL - BL = 0FEh
JA  AL_DI_ATAS        ; benar untuk tak bertanda: 255 > 1
JG  AL_LEBIH_BESAR    ; benar untuk bertanda: -1 < 1
```

Kedua lompatan di atas tidak mungkin keduanya diambil. Pada kasus ini `CF` bernilai 1 sehingga `JA` tidak diambil, sedangkan `SF` $\neq$ `OF` sehingga `JG` diambil.

Dua instruksi, dua kesimpulan berlawanan

Dari data yang sama, `JA` dan `JG` dapat memberi keputusan yang berlawanan. Tentukan lebih dahulu jenis bilangannya, lalu pilih kelompok lompatan yang sesuai.

Peta Perbandingan ke Instruksi Lompatan

Setelah `CMP dst, src`, pilih lompatan dari kolom yang sesuai dengan jenis bilangan yang dibandingkan.

Perbandingan yang diinginkan	**Bilangan bertanda**	**Bilangan tak bertanda**
$dst > src$	`JG` (*greater*)	`JA` (*above*)
$dst \geq src$	`JGE`	`JAE`
$dst < src$	`JL` (*less*)	`JB` (*below*)
$dst \leq src$	`JLE`	`JBE`
$dst = src$	`JE` / `JZ`	`JE` / `JZ`
$dst \neq src$	`JNE` / `JNZ`	`JNE` / `JNZ`

Sumber: Bab XII, Memilih Jcc yang Tepat

Tiga Kekeliruan Umum pada Percabangan

Yang paling sering muncul saat menulis percabangan

1. **Memakai `JG` untuk data tak bertanda.** Nilai seperti `FFh` dibaca sebagai negatif sehingga keputusan salah; gunakan kelompok `JA`/`JB` untuk alamat, indeks, dan pencacah.
2. **Lupa menyiapkan *flag*.** `JZ` tanpa `CMP` atau `TEST` sebelumnya akan mengikuti *flag* sisa operasi lain.
3. **Menyisipkan instruksi di antara `CMP` dan `Jcc`.** Instruksi `INC`, `DEC`, `ADD`, dan `LOOP` mengubah *flag* dan dapat merusak syarat lompatan.

Label: Aturan Penamaan dan Kesalahan Umum

Label adalah nama simbolik untuk suatu alamat di dalam segmen kode.

- Diawali huruf, garis bawah, tanda tanya, atau tanda at; tidak peka huruf besar-kecil; maksimum 31 karakter pada MASM; tidak boleh sama dengan *mnemonic*, register, atau direktif.
- Didefinisikan dengan titik dua (`SELESAI:`) tetapi ditulis tanpa titik dua sebagai operand (`JMP SELESAI`).

Tiga kesalahan berikutnya

- **Label ganda** — nama yang sama dua kali menghasilkan pesan `Symbol already defined`.
- **Alamat dilompati tanpa sengaja** — label sesudah blok keluar membuat CPU mengeksekusi byte data sebagai instruksi; letakkan `MOV AH, 4Ch` dan `INT 21h` di akhir alur.
- **Lompatan keluar jangkauan** — `Jcc` hanya *short*; baliklah syaratnya (`JNE LANJUT`) lalu pakai `JMP` *near* sebagai perantara.

Perulangan dengan Register CX

Perulangan (*loop*) adalah percabangan yang kembali ke titik awal selama syarat tertentu terpenuhi. 8086 menyediakan instruksi khusus yang memakai `CX` sebagai pencacah (*counter*).

Penting: `LOOP` mengurangi `CX` dengan satu, lalu melompat ke label bila `CX` belum nol. Karena pengujian dilakukan *setelah* pengurangan, `LOOP` dengan `CX` bernilai nol akan mengulang sebanyak 65 536 kali.

Instruksi	Aksi	Kegunaan khas
`LOOP`	$\texttt{CX} \leftarrow \texttt{CX} - 1$; lompat bila $\texttt{CX} \neq 0$	Perulangan dengan cacah yang diketahui
`LOOPE` / `LOOPZ`	Seperti `LOOP`, hanya lompat bila $\texttt{ZF} = 1$	Mencari ketidakcocokan selama nilai masih sama
`LOOPNE` / `LOOPNZ`	Seperti `LOOP`, hanya lompat bila $\texttt{ZF} = 0$	Pencarian berhenti saat nilai ditemukan
`JCXZ`	Lompat bila $\texttt{CX} = 0$; tidak mengubah `CX`	Menangani cacah nol sebelum perulangan dimulai

Sumber: Bab XII, Perulangan dengan Register CX

LOOP Dibandingkan dengan DEC dan JNZ

Perulangan yang sama dapat dibentuk dengan menggabungkan DEC dan JNZ.

Bentuk LOOP

```
    MOV CX, 5        ; ulangi lima kali
ULANG:
    ; badan perulangan
    LOOP ULANG       ; CX--; ulangi
```

Bentuk DEC + JNZ

```
    MOV CX, 5
ULANG2:
    ; badan perulangan
    DEC CX           ; kurangi pencacah
    JNZ ULANG2       ; ulangi bila <>0
```

Secara fungsional kedua bentuk itu setara. Pada 8086 asli, LOOP lebih cepat karena hanya memerlukan satu kali pengambilan instruksi. Pada prosesor generasi berikutnya, LOOP diterjemahkan ke mikrokode sehingga memerlukan lebih banyak siklus, sedangkan DEC dan JNZ dapat diprediksi dan dijalankan secara paralel; karena itu kompilator modern hampir selalu memakai pola tersebut. Pola ini juga lebih luwes karena pencacah tidak terpaku pada CX dan JNZ dapat dibuat *near*.

Rangkuman Bagian 2

- Lompatan bersyarat pada 8086 selalu berbentuk *short* dan tidak membandingkan apa pun: ia hanya membaca *flag* yang disiapkan instruksi sebelumnya.
- Kelompok bertanda (`JG`, `JGE`, `JL`, `JLE`) memakai `SF` dan `OF`; kelompok tak bertanda (`JA`, `JAE`, `JB`, `JBE`) memakai `CF` dan `ZF`; `JE`/`JNE` berlaku untuk keduanya.
- Beberapa mnemonic menunjuk opcode yang sama, misalnya `JAE`, `JNB`, dan `JNC`; nama dipilih agar maksud kode lebih jelas.
- `LOOP` mengurangi `CX` lalu melompat bila belum nol; pengujian di akhir membuat badan selalu dijalankan paling sedikit satu kali.
- Pola `DEC` dan `JNZ` setara secara fungsional, lebih luwes karena pencacah tidak harus `CX`, dan lebih cepat pada prosesor modern.

BAGIAN

Penerjemahan Pola Bahasa Tingkat Tinggi ke Assembly

Pola if

Bahasa tingkat tinggi menyediakan `if`, `while`, dan `for`; 8086 tidak mengenal satupun kata kunci tersebut. Semua struktur kontrol dibentuk dari tiga bahan dasar: pembanding yang menyiapkan *flag* (`CMP`/`TEST`), lompatan bersyarat yang memeriksanya (`Jcc`), dan lompatan tak bersyarat untuk melompati blok yang tidak dipilih (`JMP`).

jika nilai > 0 maka cetak "positif" akhir jika

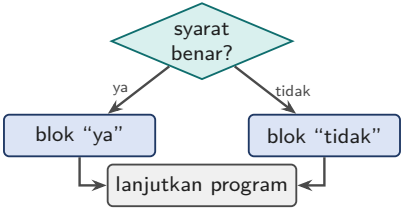
```
    CMP nilai, 0        ; siapkan flag: nilai - 0
    JLE LEWATI          ; bila nilai <= 0, lompati blok
    MOV AH, 09h         ; cetak string "positif"
    LEA DX, pesan_pos
    INT 21h
LEWATI:                 ; lanjutkan program
```

Kunci pola ini

Bahasa tingkat tinggi menuliskan "kerjakan blok bila syarat benar", sedangkan assembly menuliskan "lompati blok bila syarat *tidak* benar" — syaratnya dibalik.

Pola if–else

Bila terdapat cabang alternatif, diperlukan **dua** lompatan: satu untuk memilih cabang, satu untuk melompati cabang lainnya.




```
    MOV AX, a
    CMP AX, b         ; bandingkan a, b
    JNE CABANG_BEDA   ; ke cabang lain
    MOV AH, 09h       ; cabang "sama"
    LEA DX, pesan_sama
    INT 21h
    JMP GABUNG        ; lompati cabang else
CABANG_BEDA:
    MOV AH, 09h       ; cabang "beda"
    LEA DX, pesan_beda
    INT 21h
GABUNG:
```

`JMP GABUNG` wajib ada; tanpa instruksi itu, setelah mencetak “sama” prosesor akan menjatuhi (*fall through*) blok “beda” dan mencetak keduanya.

Pola while, do–while, dan for

Pola	Bentuk bahasa tingkat tinggi	Padanan assembly
`while`	`while (kond) { A }`	`uji`: uji di awal; lompat keluar bila salah; `A`; `JMP` kembali ke `uji`
`do-while`	`do { A } while (kond);`	badan: `A`; uji di akhir; lompat kembali bila benar (dijalankan minimal sekali)
`for`	`for (i=0; i<n; i++) { A }`	inisialisasi `CX`; `ulang`: `A`; `INC`/`ADD` pencacah; `LOOP` atau `DEC`+`JNZ`

`while` menguji syarat di awal sehingga badan dapat dijalankan nol kali, sedangkan `do-while` menguji di akhir sehingga badan selalu dijalankan paling sedikit satu kali. Karena itu `while` memerlukan `JMP` tambahan untuk kembali ke titik pengujian, sedangkan `do-while` cukup memakai `LOOP` atau `JNZ`.

Penting: Bila pencacah harus naik dari nol, `LOOP` tidak dapat dipakai langsung karena `LOOP` selalu mengurangi.

```
    MOV SI, 0           ; i = 0
ULANG_NAIK:
    INC SI              ; i++
    CMP SI, 8           ; i < 8 ?
    JB  ULANG_NAIK      ; lanjut bila masih di bawah batas
```

Pola switch dengan Tabel Lompatan

`switch` dengan banyak cabang tidak efisien bila diterjemahkan sebagai rangkaian `CMP` dan `Jcc` berantai. Alternatifnya adalah **tabel lompatan** (*jump table*): tabel berisi alamat setiap cabang, diindeks dengan nomor kasus.

```
.DATA
    TABEL DW OFFSET KASUS0, OFFSET KASUS1, OFFSET KASUS2
.CODE
    MOV SI, nomor        ; SI = nomor kasus (0, 1, atau 2)
    CMP SI, 2            ; periksa batas atas
    JA  DI_LUAR_BATAS    ; nomor di luar jangkauan ditolak
    SHL SI, 1            ; setiap entri berukuran 2 byte
    JMP TABEL[SI]        ; lompat ke alamat yang tersimpan pada tabel
```

- Indeks harus dikalikan ukuran entri: karena `DW`, indeks digeser satu bit ke kiri dengan `SHL`.
- Batas atas *wajib* diperiksa sebelum pengindeksan; tanpa `CMP SI, 2` dan `JA`, nomor kasus yang salah melompat ke alamat acak.

Studi Kasus: Maksimum, Gelembung, dan Pencarian Biner

Kasus 1: mencari nilai maksimum

Elemen pertama dianggap kandidat maksimum, lalu dibandingkan dengan setiap elemen berikutnya dan diperbarui bila ada yang lebih besar. Perbandingan memakai `CMP` diikuti `JAE` (tak bertanda); kompleksitasnya $O(n)$.

Kasus 2: pengurutan gelembung

Larik diproses berulang kali; pada setiap lintasan dua elemen bersebelahan dibandingkan dan ditukar bila urutannya salah, sehingga elemen terbesar bergerak ke posisi paling kanan. Perulangan bersarang diterjemahkan menjadi dua pencacah, sehingga `CX` lintasan luar disimpan sementara di *stack* dengan `PUSH` dan `POP`. Untuk n elemen, sekitar $n^2/2$ pembandingan.

Kasus 3 (pengantar): pencarian biner

Pada larik yang *sudah terurut*, batas bawah dan batas atas disimpan pada dua register penunjuk, dan titik tengah dihitung dengan `ADD` lalu `SHR AX, 1`. Paling banyak $\log_2 n$ pembandingan.

Rangkuman Bagian 3

- Tidak ada struktur kontrol baru pada tingkat mesin: yang ada hanyalah pembanding, lompatan bersyarat, dan lompatan tak bersyarat.
- `if` diterjemahkan dengan membalik syarat: lompati blok bila syarat *tidak* benar.
- `if-else` memerlukan dua lompatan; `JMP` pengakhir cabang "ya" wajib ada agar blok "tidak" tidak dijatuhi.
- `while` menguji di awal (badan boleh nol kali), `do-while` menguji di akhir (badan minimal sekali); `for` dengan pencacah turun paling mudah memakai `CX` dan `LOOP`.
- `switch` dengan banyak cabang paling efisien memakai tabel lompatan: indeks dikalikan ukuran entri dan batas atas wajib diperiksa lebih dahulu.

Contoh 1: Mencari Nilai Maksimum Larik

`loop-maksimum.asm` mencari nilai terbesar dalam sebuah larik 16-bit.

```
    LEA SI, deret          ; SI = awal larik
    MOV CX, BANYAK - 1     ; n - 1 banding
    MOV AX, [SI]           ; kandidat elemen 1
    ADD SI, 2              ; maju satu word
CARI:
    CMP AX, [SI]           ; siapkan flag
    JAE LANJUT             ; kandidat >= elemen
    MOV AX, [SI]           ; kandidat baru
LANJUT:
    ADD SI, 2
    LOOP CARI              ; CX--; ulangi
```

- `MOV CX, BANYAK - 1`: elemen pertama sudah menjadi kandidat, sehingga hanya $n - 1$ elemen diperiksa.
- `CMP AX, [SI]` menyiapkan *flag*, lalu `JAE` — bukan `JGE` — mengambil keputusan karena larik diperlakukan sebagai bilangan tak bertanda.

Keluaran

```
Nilai maksimum: 93
```

Contoh 2: Pengurutan Gelembung

Perulangan luar dan dalam

```
LINTASAN:
    PUSH CX              ; simpan pencacah
    MOV CX, BANYAK - 1   ; pencacah dalam
BANDING:
    LOOP BANDING
    POP CX               ; ambil pencacah
    LOOP LINTASAN
```

Inti pembandingan dan penukaran

```
    MOV SI, 0            ; indeks kiri
    MOV DI, 1            ; indeks kanan
    MOV AL, deret[SI]    ; elemen kiri
    MOV BL, deret[DI]    ; elemen kanan
    CMP AL, BL           ; siapkan flag
    JBE TIDAK_TUKAR      ; sudah urut
    MOV deret[SI], BL    ; tukar
    MOV deret[DI], AL
TIDAK_TUKAR:
    INC SI
    INC DI
```

`bubble-sort.asm` mengurutkan larik delapan byte secara menaik; perulangan luar dijalankan $n - 1 = 7$ kali. Karena perulangan dalam memakai register `CX` yang sama, pencacah luar harus diselamatkan dengan `PUSH CX`; tanpa itu `LOOP` luar menguranginya menjadi `FFFFh` dan perulangan berjalan tanpa henti.

Keluaran: `Hasil pengurutan:  1 2 3 4 6 7 8 9`

Praktikum: Pencarian Maksimum dan Penelusuran Alur

Judul praktikum

Pencarian maksimum, pengurutan sederhana, dan penelusuran alur percabangan ▪ **Sub-CPMK:** 3.3 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

Alat dan bahan

- Emu8086, GUI Turbo Assembler, dan DOSBox
- Berkas `loop-maksimum.asm` dan `bubble-sort.asm`
- Lembar kerja penelusuran

Langkah kerja

1. Rakit dan jalankan `loop-maksimum.asm`.
2. Ubah larik menjadi $5, 4, 3, 2, 1, 0, \texttt{FFh}, 1$; tentukan maksimumnya sebagai bilangan tak bertanda.
3. Hapus `PUSH CX` dan `POP CX`; jelaskan yang berubah.
4. Dengan *Single Step*, catat `ZF`, `CF`, `SF`, dan `OF` sebelum dan sesudah `CMP AX, [SI]`; lalu ganti `LOOP CARI` dengan `DEC CX` dan `JNZ CARI` dan bandingkan ukuran `.obj`.

Dikumpulkan (Tugas 5): program modifikasi, tangkapan layar, lembar kerja penelusuran, dan jurnal praktikum.

Latihan

1. **Konseptual.** Jelaskan mengapa lompatan bersyarat pada 8086 tidak dapat membandingkan dua bilangan secara langsung; sebutkan dua instruksi penyiap *flag* dan perbedaannya.
2. **Analisis *flag*.** Diberikan potongan berikut; tentukan `ZF`, `CF`, `SF`, dan `OF` setelah `CMP`, lalu tentukan manakah yang diambil: `JA` atau `JB`, dan `JG` atau `JL`? Jelaskan mengapa kedua jawabannya berbeda.

```
MOV AL, 0FEh
MOV BL, 03h
CMP AL, BL
```

3. **Pemilihan `Jcc`.** Tuliskan lompatan yang tepat untuk: `AX` $>$ `BX` (tak bertanda); `DL` $\leq$ `7Fh` (tak bertanda); `SI` $=$ `DI`; hasil operasi nol dan tidak terjadi limpahan; `AL` berisi bilangan genap.
4. **Prediksi alur.** Perulangan bersarang dengan `MOV CX, 3` di lintasan luar dan `MOV CX, 2` di lintasan dalam, dilengkapi `PUSH CX` dan `POP CX`: berapa kali `INC BX` dijalankan, dan apa yang terjadi bila kedua instruksi *stack* itu dihapus?
5. **Analisis kesalahan.** `JNE SELESAI` diikuti 200 byte instruksi lain menghasilkan pesan `Relative jump out of range`. Jelaskan penyebabnya dan tuliskan perbaikan yang memakai syarat terbalik serta `JMP` *near*.

Rangkuman

- Pengambilan keputusan berjalan dalam dua langkah: satu instruksi menyiapkan *flag*, lalu satu lompatan bersyarat membacanya. Lima *flag* terpenting adalah `ZF`, `CF`, `SF`, `OF`, dan `PF`.
- `CMP` menyiapkan *flag* melalui pengurangan yang hasilnya dibuang, sedangkan `TEST` melalui `AND` yang hasilnya dibuang. `CMP` membandingkan nilai; `TEST` memeriksa bit.
- `JMP` memindahkan kendali tanpa syarat. Jarak lompatan menentukan ukuran instruksi: *short* dua byte ($\pm$ 127 byte), *near* tiga byte (satu segmen), dan *far* lima byte (antarsegmen).
- Keluarga `Jcc` terbagi menjadi kelompok bertanda (`JG`, `JGE`, `JL`, `JLE`), kelompok tak bertanda (`JA`, `JAE`, `JB`, `JBE`), pemeriksa *flag* tunggal (`JC`, `JS`, `JO`, `JP`, `JCXZ`), dan kelompok kesamaan (`JE`/`JNE`) yang berlaku untuk keduanya.
- Perulangan dibentuk dengan `LOOP` dan `CX`, varian `LOOPE`/`LOOPNE` yang menambahkan syarat `ZF`, atau pola `DEC` dan `JNZ` yang lebih luwes dan lebih cepat pada prosesor modern.
- Seluruh struktur kontrol bahasa tingkat tinggi — `if`, `while`, `for`, dan `switch` — diterjemahkan dari tiga bahan dasar: `CMP`/`TEST`, `Jcc`, dan `JMP`. `switch` dengan banyak cabang paling efisien memakai tabel lompatan.

Cek Pemahaman

1. Instruksi yang menyiapkan *flag* melalui pengurangan tanpa menyimpan hasilnya adalah ...
   - 1.1 `SUB`
   - 1.2 `CMP`
   - 1.3 `TEST`
   - 1.4 `AND`
2. Setelah `CMP AX, BX`, lompatan yang tepat untuk `AX` $<$ `BX` (tak bertanda) adalah ...
   - 2.1 `JL`
   - 2.2 `JG`
   - 2.3 `JB`
   - 2.4 `JA`
3. Instruksi `LOOP` bekerja dengan cara ...
   - 3.1 mengurangi `CX`, lalu melompat bila belum nol
   - 3.2 menambah `CX`, lalu melompat bila belum nol
   - 3.3 menguji `CX` lebih dahulu, lalu melompat
   - 3.4 menguji `ZF` tanpa mengubah `CX`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan praktikum (Tugas 5): kedua program versi modifikasi beserta tangkapan layar, lembar kerja penelusuran, jawaban langkah 1–4, dan jurnal praktikum.
- Ulangi bagian yang belum dikuasai dengan menelusuri `loop-maksimum.asm` satu instruksi demi satu pada *Single Step*.
- Baca **Bab XIII** — Prosedur dan Manajemen Stack.

Bacaan lanjutan: Bab XII bagian Memilih `Jcc` yang Tepat, dan Pola `switch` dengan Tabel Lompatan; pembahasan register `FLAGS` pada Bab V; pemakaian *stack* pada Bab XIII.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil